

Intro to Algorithms Notes

Based on Cormen et al.

Abdullah Yassine

June 22, 2025

Contents

2	Chapter 2: Getting Started	2
2.1	Insertion Sort	2
2.2	Analyzing Algorithms	3
2.3	Designing Algorithms	4
2.3.1	The divide-and-conquer approach	4

2 Chapter 2: Getting Started

2.1 Insertion Sort

We begin our studies by first learning the tools and frameworks we will be using in designing and analyzing algorithms. We do this by first describing and solving the **sorting problem**.

Definition 2.1. *Sorting Problem*

Sorting Problem Definition

input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$

output: A permutation or reordering $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$

Definition 2.2. *The keys are the numbers that we wish to sort.*

The **pseudo-code** for the **insertion-sort** algorithm is provided below. You can find the python implementation in the following path `./ch3.ipynb` of the current working directory.

Algorithm 1 INSERTION-SORT(A)

Require: Array $A[1 \dots n]$

Ensure: Sorted array A in non-decreasing order

```

1: for  $j = 2$  to  $n$  do
2:    $key \leftarrow A[j]$ 
3:    $i \leftarrow j - 1$ 
4:   while  $i > 0$  and  $A[i] > key$  do
5:      $A[i + 1] \leftarrow A[i]$                                 ▷ Shift element right
6:      $i \leftarrow i - 1$ 
7:   end while
8:    $A[i + 1] \leftarrow key$                                 ▷ Insert key in position
9: end for
```

You can see that the algorithm first assumes that the first index is sorted. It then takes the next index (just like how you would take the next card out of the cards you have) and sort it in the right place by placing the card in the appropriate place. It does this for all the cards.

As you can see, the insertion-sort algorithm sorts the list **in-place**.

So, how does this algorithm work? We start off by knowing that the subarray $A[1, j - 1]$ is already sorted.

Which means that now we have j that we need to sort which means that the subarray $A[j + 1, n]$ is the one to yet be sorted.

2.2 Analyzing Algorithms

Before we start explaining how algorithms analyzed, we need to define a few things. First is the **input size** which is different from one problem to another. For example, in the case of sorting and more specifically INSERTION-SORT, the input size is the size of the array we are dealing with.

Another thing is **running time** which is the number of steps or operations executed.

We won't go over analyzing INSERTION-SORT here but we will define some terms though. First is the fact that **worst-case running time** means the *longest* running time for any n . Sometimes we should be interested in **average-case**, but what constitutes "average" is limited. In this book, we can assume that any possibility given an n is equally possible. Sometimes, we can use **randomized algorithm** to randomize the input that the algorithm receives and yield an **expected** time out of that.

Thus, for INSERTION-SORT, for the worst-time, if we ignore lower-order terms and leading coefficients, we are left with n^2 . Thus, INSERTION-SORT has a worst-case running time of $\Theta(n^2)$.

Generally, we consider one algorithm to be better than the other if the worst-time is smaller than the other. For small inputs, however, sometimes the one with higher worst-time can outperform the one with the smaller worst-time. In a lot of cases, we assume we are using big input sizes.

2.3 Designing Algorithms

In this subsection, we will introduce **divide-and-conquer**.

2.3.1 The divide-and-conquer approach

We introduce **divide-and-conquer** which is the idea that we can divide a problem into smaller sub-problems that have the same problem but it is in smaller size. We solve those problems and combine these solutions to solve the original problem **recursively**.

The divide-and-conquer algorithm follows the following three steps:

1. **Divide** the problem into smaller problems that are smaller instances of the same problem.
2. **Conquer** the subproblems by solving them recursively.
3. **Combine** the solutions to the subproblems into the solution for the original problem.

The **MERGE-SORT** algorithm follows this same approach, and works as follows:

1. **Divide** the n -element sequence to be sorted into two subsequences of $n/2$ elements each.
2. **Conquer** by sorting the sub-sequences recursively using merge sort.
3. **Combine** by merging the two sorted subsequences to produce the sorted answer.